# POS Backend Guide

How the system works, every API endpoint and why it exists, and what to build next to make it faster and stand out

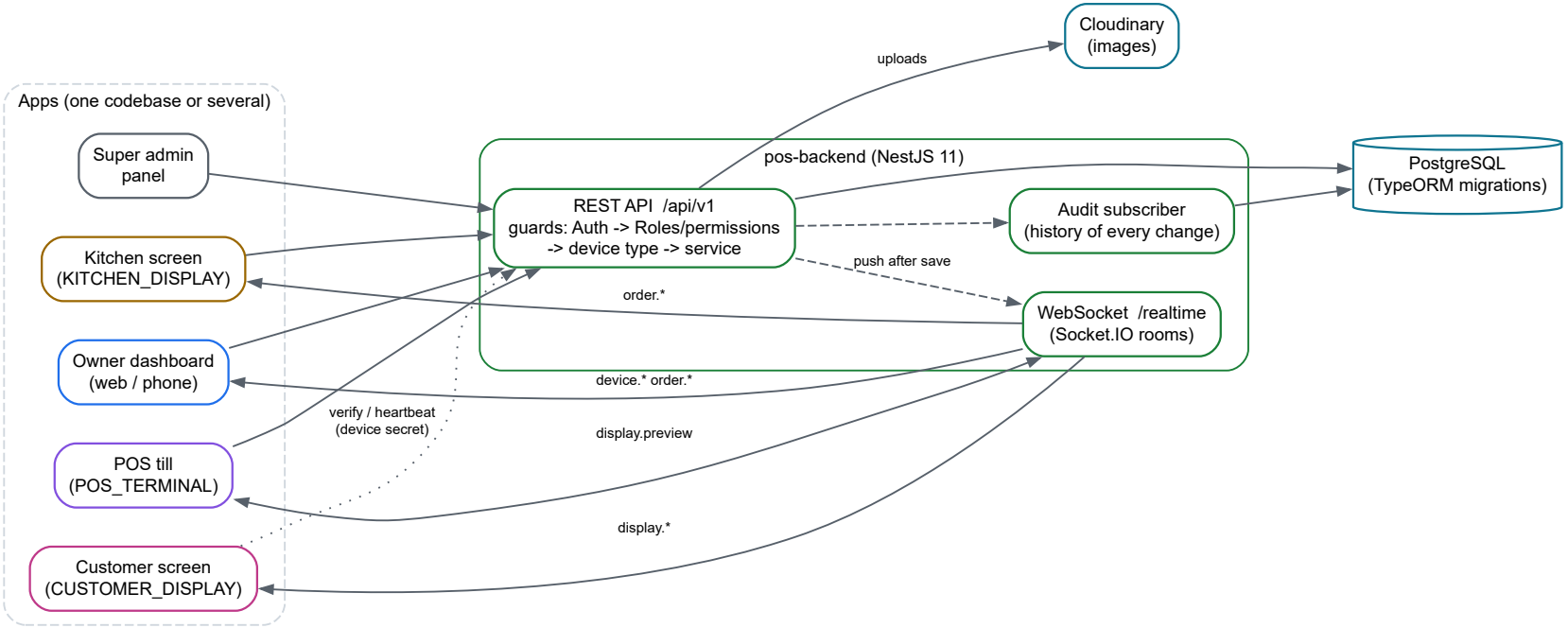
| 92 | 8 | 8 | 4 |
|---|---|---|---|
| REST endpoints | realtime event groups | workflows | device / app types |


1. Architecture
2. How every request is checked
3. Roles and permissions
4. Workflows
5. API reference
6. Realtime events
7. What already makes it strong
8. Good to have: speed and safety
9. Future features
10. Suggested roadmap


Generated 2026-10-30 from the pos-backend source (NestJS 11, TypeORM, PostgreSQL, Socket.IO). Base URL: `/api/v1` · Swagger: `/api/v1/doc` · Companion document: POS-Database-Schema.pdf

# 1. Architecture

One NestJS server serves all apps. Apps change data through the REST API; the server pushes changes back over one WebSocket so every screen stays current without refreshing. All data lives in PostgreSQL; images go to Cloudinary.




## Who uses it

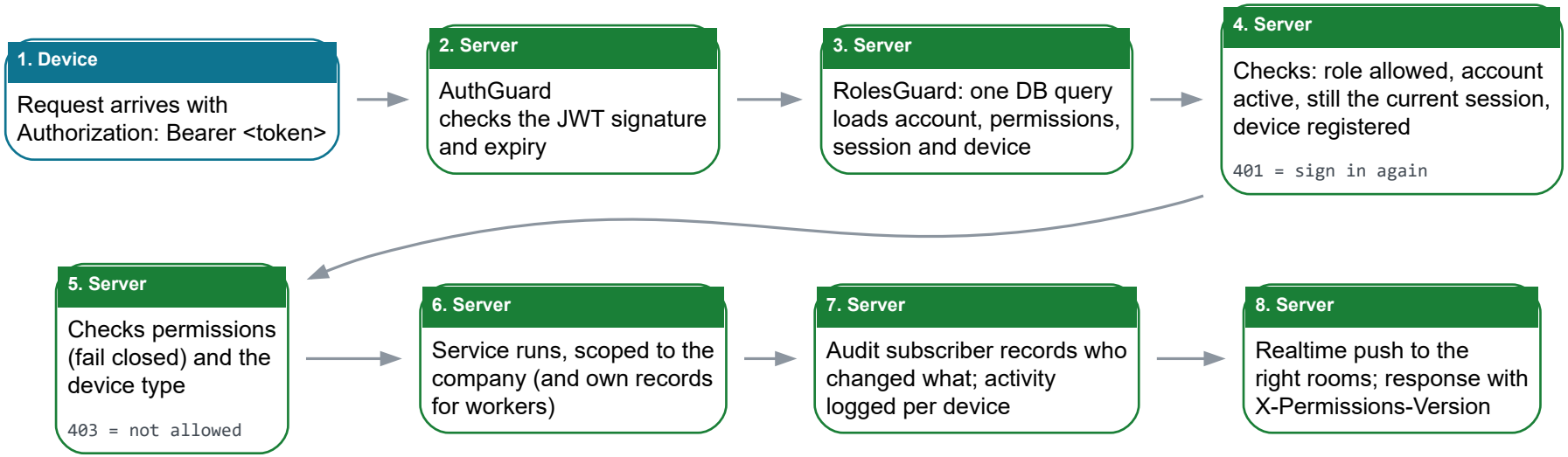
- **Super admin**: creates businesses and their owners.
- **Owner**: runs a business: catalog, staff, devices, reports. Signs in with email + password.
- **Worker**: sells, cooks, manages what their role allows. Signs in with a PIN on an approved device.
- **Devices**: till, kitchen screen, customer screen, each with its own job.

## Main building blocks

- **Modules**: auth/users, companies, devices, employees and roles, catalog, files, carts and items, orders and payments, floor plan, audit, realtime.
- **Guards**: AuthGuard (token) and RolesGuard (role, permissions, session, device, device type).
- **Audit subscriber**: records every insert / update / delete with who did it.
- **Realtime gateway**: Socket.IO namespace `/realtime` with rooms per user, device, company, orders, devices and customer display.

# 2. How every request is checked

Every protected request goes through the same steps. Permissions are read live from the database (not from the token), so changes apply at once. Anything not explicitly allowed for workers is refused ("fail closed").




### Status codes the apps should handle

- **401**: sign in again (token expired, signed in on another device, device disabled, account deactivated).
- **403**: signed in but not allowed (missing permission, wrong device type, e.g. "Not available on a kitchen display").
- **404**: not found *or* belongs to another company (never reveals other companies' data).
- **409**: already exists (e.g. device already registered). **429**: too many wrong logins.

### Tokens and devices

- Access token (JWT, currently 1 day) carries user, role, company, device and session id.
- Workers: one active session at a time unless allowed several devices; a new PIN login ends the old one.
- Devices prove themselves with a secret given once at registration (heartbeat, socket).
- Every request from a device updates its "last seen" and activity log.

# 3. Roles and permissions

## Account roles

| Role | Can do |
|---|---|
| **SUPERADMIN** | Everything, every company. |
| **OWNER** | Everything in their own company. |
| **EMPLOYEE** | Only what their role's permissions (plus personal extras, minus removals) allow; only their own carts and orders unless given orders.view_all. |

## Delegation rules (no escalation)

- A worker with employees.manage / roles.manage can manage staff and roles, but:
- cannot give a permission they do not have themselves;
- cannot change their own access or role;
- cannot manage someone (or a role) with more access than they have;
- cannot create an "everything" role. Only owners have unlimited access.

## Permission keys

| Key | Unlocks | Status |
|---|---|---|
| `sales.create` | Open carts, add items, move order status | Built |
| `sales.checkout` | Take payment and complete orders | Built |
| `sales.split_payment` | Pay one order with several methods | Built |
| `sales.credit` | Let a customer pay later (credit) | Built |
| `sales.cancel` | Cancel orders | Built |
| `orders.view_own / view_all` | See own orders / all company orders | Built |
| `tables.manage` | Change room and seat status | Built |
| `products.manage` | Products, categories, their images | Built |
| `reports.view` | History and staff performance | Built |
| `employees.manage` | Workers, PINs, status | Built |
| `roles.manage` | Roles and per-worker permissions | Built |
| `devices.manage` | Approve and configure devices | Built |
| `settings.manage` | Company profile | Built |
| `inventory.view / inventory.adjust` | Stock levels and adjustments | Reserved: no endpoints yet |
| `credits.view` | List of customers who owe money | Reserved: no endpoints yet |
| `finance.verify` | Verify cash-up / reconcile money | Reserved: no endpoints yet |
| `services.manage` | Services (e.g. hotel / salon services) | Reserved: no endpoints yet |

Some keys imply others (e.g. sales.checkout includes sales.create; orders.view_all includes view_own).

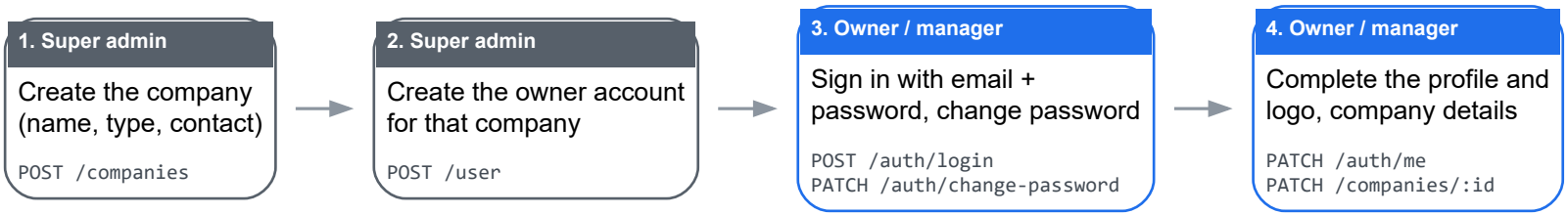
# 4. Workflows

Each workflow shows the steps in order, who does them, and the endpoint or live event involved.

Super admin · Owner / manager · Worker · Device · Server · Customer display · Kitchen display

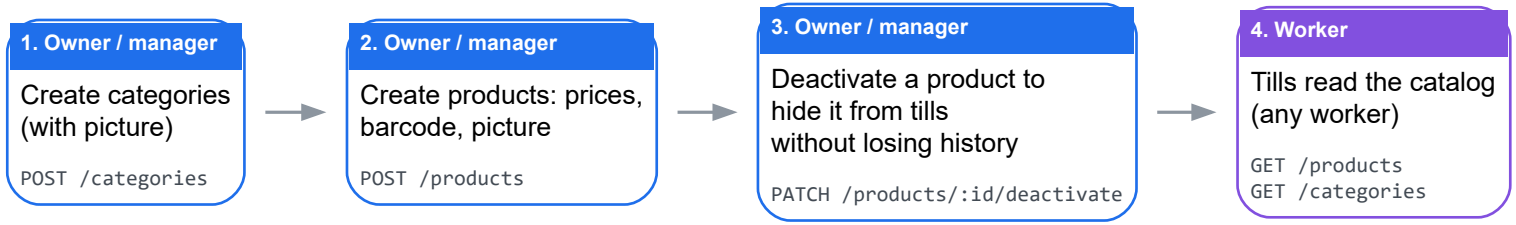
## Onboarding a business

Only the platform operator creates businesses, so every company and owner is known and nobody can sign up into another company.



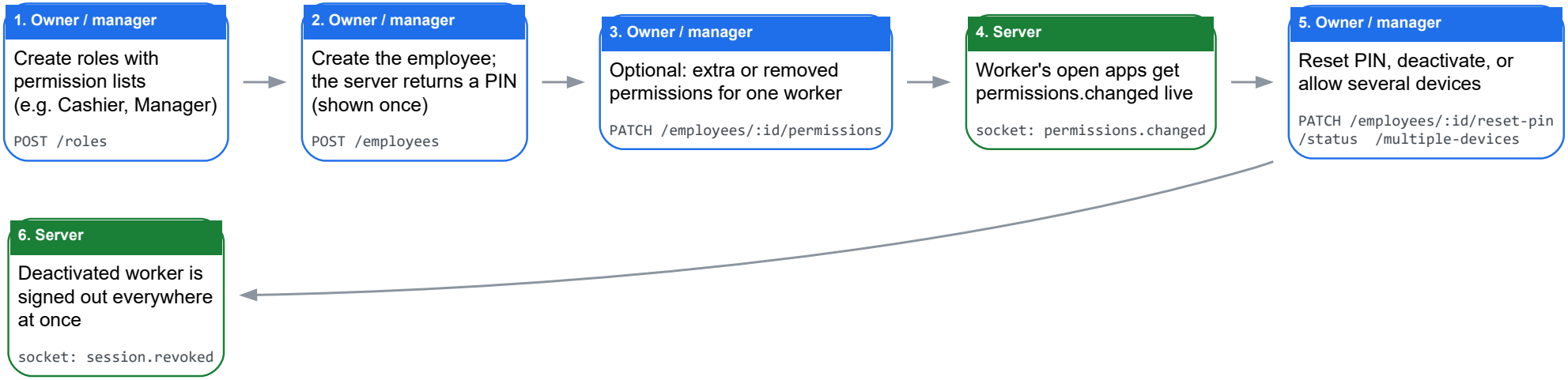

## Setting up what is sold

Products belong to one company; names and barcodes are unique per company so scanning always finds exactly one product.



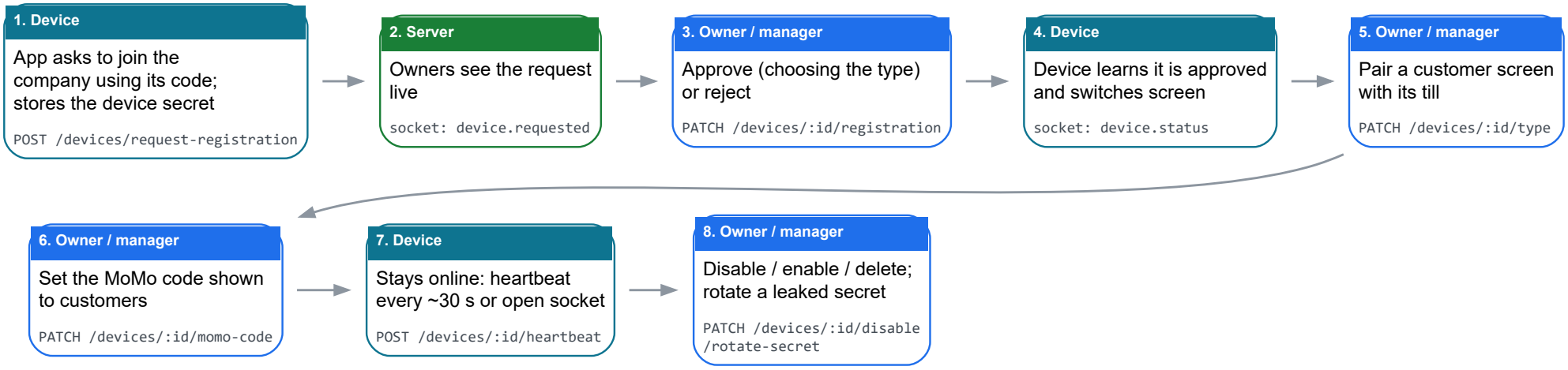
## Staff, roles and permissions

Owners decide exactly what each worker may do. Changes apply instantly to signed-in workers (no re-login) and nobody can give access they do not have.



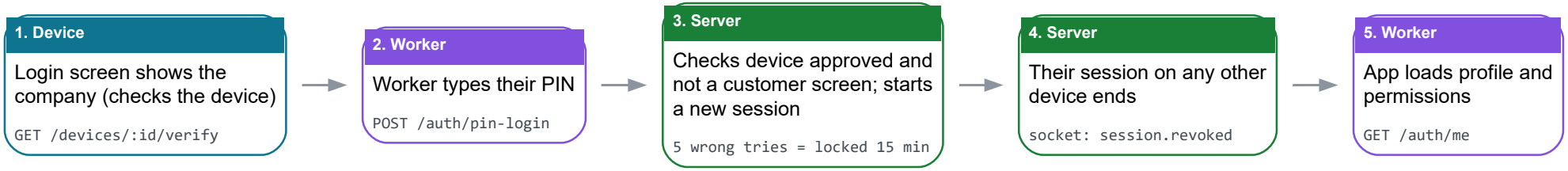

## Registering and setting up a device

Only devices the owner approved can be used. Each device has a secret, a type (till, kitchen, customer screen) and optionally its own MoMo code.



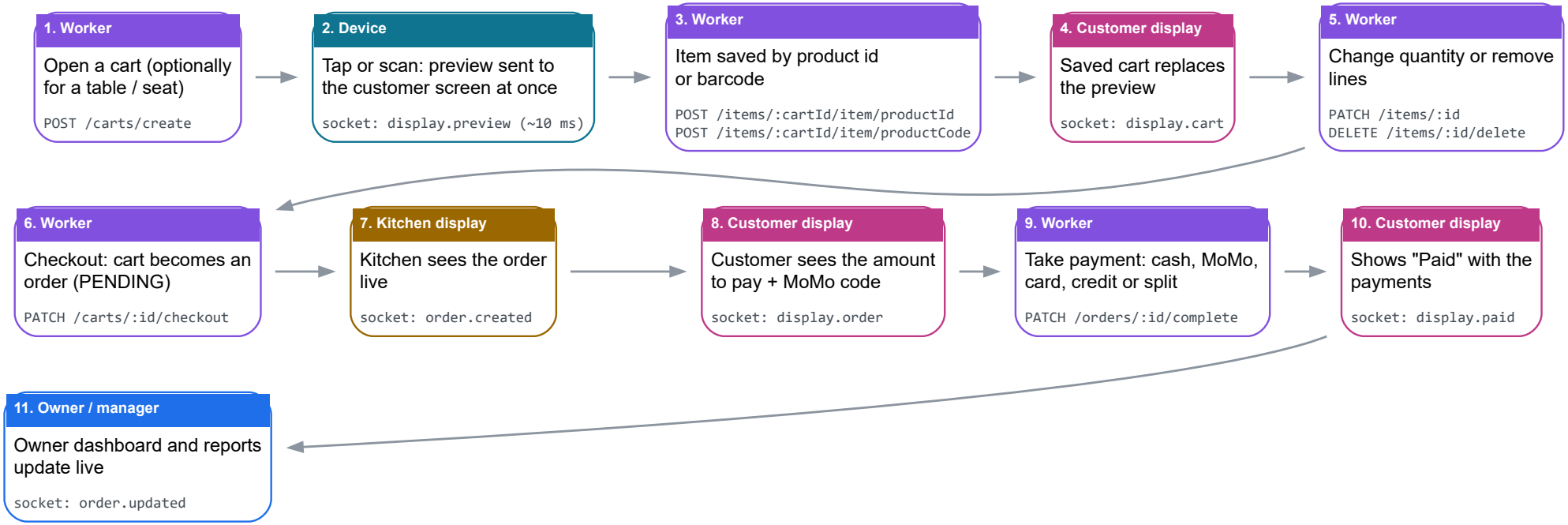
## Worker sign-in on a till

Workers use a short PIN on an approved device. A worker is signed in on one device at a time (unless allowed more), so a PIN cannot be shared quietly.




## A sale, from scan to paid

The core loop of the product. It is built so the customer screen reacts instantly, totals are exact to the cent, and every step is recorded.



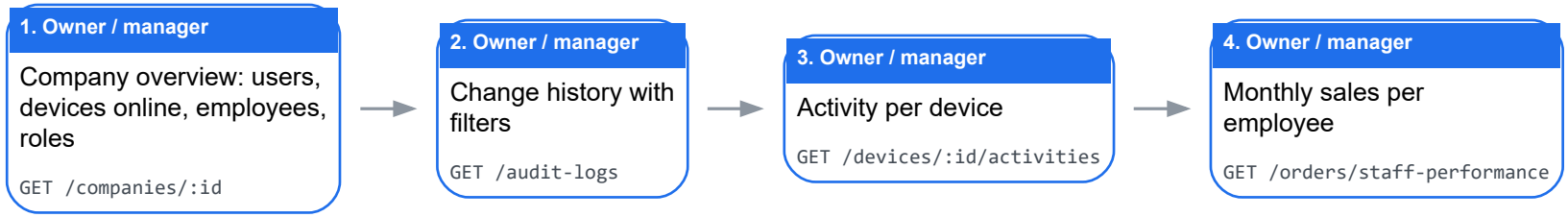
## Kitchen workflow

Kitchen screens can follow and move orders but cannot sell or take money, so a kitchen device cannot be misused as a till.




## Oversight: history and reports

Owners can always answer "who did this and when". Every create, change and delete is recorded with the person and the before / after values.



# 5. API reference

All paths start with `/api/v1`. Send `Authorization: Bearer <accessToken>` except on Public endpoints. "Who" is the role or permission needed; "till" / "kitchen" means the device type it must be called from. Uploads use `multipart/form-data` (field `picture` or `logo`).

## Authentication and profile

| | Endpoint | Who | What it does and why |
|---|---|---|---|
| POST | `/auth/login` | Public | Owners and admins sign in with email and password. Returns an access token. Wrong passwords are limited (5 tries, then 15 minutes). |
| POST | `/auth/pin-login` | Public | Workers sign in on an approved device with their PIN. Ends their session on other devices (single-device rule). Refused on customer displays. |
| POST | `/auth/logout` | Signed in | Ends the session and frees the device. |
| PATCH | `/auth/change-pin` | Worker | A worker replaces the PIN they were given. |
| PATCH | `/auth/change-password` | Signed in | Change own password (also used for the forced first change). |
| GET | `/auth/me` | Signed in | Who am I: profile, company, role and the effective permissions. Apps refetch it when X-Permissions-Version changes. |
| PATCH | `/auth/me` | Signed in | Update own name, phone, email and picture. |

## Users (super admin)

| | Endpoint | Who | What it does and why |
|---|---|---|---|
| POST | `/user` | Super admin | Create an owner or employee account for a company (returns a PIN for workers). |
| GET | `/user` | Super admin | List all users on the platform. |
| GET | `/user/:userId` | Super admin | One user. |
| DELETE | `/user/:userId` | Super admin | Remove a user. |

## Companies

| | Endpoint | Who | What it does and why |
|---|---|---|---|
| POST | `/companies` | Super admin | Create a business (gets a code devices register with). |
| GET | `/companies` | Super admin | All businesses. |
| GET | `/companies/:companyId` | settings.manage | Everything about a company in one call: users, devices (online, who is signed in, last activity), employees and roles. |

| | Endpoint | Who | What it does and why |
|---|---|---|---|
| PATCH | `/companies/:companyId` | settings.manage | Edit the company profile and logo. Owners only their own company. |
| DELETE | `/companies/:companyId/delete` | Super admin | Delete a business (cascades to its sales and catalog). |

## Devices

| | Endpoint | Who | What it does and why |
|---|---|---|---|
| GET | `/devices/:deviceId/verify` | Public | Is this device approved? Returns company name, logo and the MoMo code for the login / customer screen. |
| POST | `/devices/:deviceId/heartbeat` | Device secret | Keeps a device "online" while idle (every ~30 s). |
| POST | `/devices/request-registration` | Public | A new device asks to join a company. Returns the device secret once. |
| PATCH | `/devices/:deviceId/registration` | devices.manage | Approve (optionally with its type) or reject a waiting device. |
| PATCH | `/devices/:deviceId/type` | devices.manage | Make it a till, kitchen screen or customer screen; pair a customer screen with a till. Applies live. |
| PATCH | `/devices/:deviceId/momo-code` | devices.manage | MoMo pay code for this device. A customer screen without one shows its till's. |
| PATCH | `/devices/:deviceId/rotate-secret` | devices.manage | New device secret if the old one leaked (shown once). |
| PATCH | `/devices/:deviceId/enable` | devices.manage | Allow a disabled device again. |
| PATCH | `/devices/:deviceId/disable` | devices.manage | Block a device; whoever is on it is signed out at once. |
| GET | `/devices` | devices.manage | All devices with status, type, online flag and current user (filter by status, e.g. PENDING). |
| GET | `/devices/:deviceId/activities` | devices.manage | What was done on the device and by whom. |
| GET | `/devices/:deviceId` | devices.manage | One device. |
| DELETE | `/devices/:deviceId/delete` | devices.manage | Remove a device; it must register again. |

## Roles and employees

| | Endpoint | Who | What it does and why |
|---|---|---|---|
| POST | `/roles` | roles.manage | Create a role from permission keys. Cannot include access you do not have. |
| GET | `/roles` | roles.manage | List roles. |
| GET | `/roles/:roleId` | roles.manage | One role. |
| PATCH | `/roles/:roleId` | roles.manage | Change a role; everyone with it is updated live. |
| DELETE | `/roles/:roleId/delete` | roles.manage | Delete a role. |

| | Endpoint | Who | What it does and why |
|---|---|---|---|
| POST | `/employees` | employees.manage | Add a worker (with picture); creates their sign-in and returns the PIN once. |
| GET | `/employees` | employees.manage | List workers. |
| GET | `/employees/:employeeId` | employees.manage | One worker. |
| PATCH | `/employees/:employeeId` | employees.manage | Edit a worker, change role or picture. |
| PATCH | `/employees/:employeeId/reset-pin` | employees.manage | New PIN for a worker who forgot it (shown once). |
| PATCH | `/employees/:employeeId/status` | employees.manage | Activate / deactivate. Deactivated workers are signed out everywhere instantly. |
| PATCH | `/employees/:employeeId/multiple-devices` | employees.manage | Allow a worker to be signed in on several devices at once. |
| PATCH | `/employees/:employeeId/permissions` | employees.manage + roles.manage | Give or take away single permissions for one worker on top of the role. |
| DELETE | `/employees/:employeeId/delete` | employees.manage | Remove a worker. |

## Catalog (categories and products)

| | Endpoint | Who | What it does and why |
|---|---|---|---|
| POST | `/categories` | products.manage | Create a category with picture. |
| GET | `/categories` | Any worker | Categories for the till. |
| GET | `/categories/:categoryId` | Any worker | One category. |
| PATCH | `/categories/:categoryId` | products.manage | Rename or change picture. |
| DELETE | `/categories/:categoryId/delete` | products.manage | Delete a category. |
| POST | `/products` | products.manage | Create a product: buying / selling price, barcode, picture. |
| GET | `/products` | Any worker | The catalog for the till. |
| GET | `/products/:productId` | Any worker | One product. |
| PATCH | `/products/:productId` | products.manage | Edit prices, barcode, picture. |
| PATCH | `/products/:productId/deactivate` | products.manage | Hide from sale, keep in history. |
| PATCH | `/products/:productId/activate` | products.manage | Sell it again. |

## Files

| | Endpoint | Who | What it does and why |
|---|---|---|---|
| DELETE | `/file/:id` | Owner of the image | Delete an image (also from Cloudinary). Workers only their own avatar unless they manage products / employees. |

## Carts

| | Endpoint | Who | What it does and why |
|---|---|---|---|
| POST | `/carts/create` | sales.create, till | Start a sale (optionally for a seat). |
| GET | `/carts` | sales.create, till | Open carts of this worker (e.g. after a restart). |
| PATCH | `/carts/:cartId/checkout` | sales.create, till | Turn the cart into an order to pay; kitchen and customer screen are notified. |
| DELETE | `/carts/:cartId/abandon` | sales.create, till | Drop the cart; the customer screen clears. |

## Cart items

| | Endpoint | Who | What it does and why |
|---|---|---|---|
| POST | `/items/:cartId/item/productId` | sales.create, till | Add a product by tapping it. Returns the full cart; customer screen updated. |
| POST | `/items/:cartId/item/productCode` | sales.create, till | Add a product by scanning its barcode. |
| PATCH | `/items/:itemId` | sales.create, till | Change quantity. |
| DELETE | `/items/:itemId/delete` | sales.create, till | Remove a line. |

## Orders and payments

| | Endpoint | Who | What it does and why |
|---|---|---|---|
| GET | `/orders` | orders.view_own (till / kitchen) | Orders: own ones, or all of the company with orders.view_all. |
| GET | `/orders/staff-performance` | reports.view | Monthly completed-sales totals per employee for a year. |
| GET | `/orders/:orderId` | orders.view_own (till / kitchen) | One order with items and payments. |
| PATCH | `/orders/:orderId/status` | sales.create (till / kitchen) | Move an order to PREPARING / READY. |
| PATCH | `/orders/:orderId/complete` | sales.checkout, till | Record payments (must equal the total to the cent). Split needs sales.split_payment; credit needs sales.credit. |
| PATCH | `/orders/:orderId/cancel` | sales.cancel, till | Cancel an open order. |

## History

| | Endpoint | Who | What it does and why |
|---|---|---|---|
| GET | `/audit-logs` | reports.view | Change history: who created / updated / deleted what and when, with before / after values. |

## Floor maps

| | Endpoint | Who | What it does and why |
|---|---|---|---|
| POST | `/floor-maps` | Owner | Create a floor. |
| GET | `/floor-maps` | Any worker | Floors. |
| GET | `/floor-maps/:floorMapId` | Any worker | One floor with its rooms. |
| PATCH | `/floor-maps/:floorMapId` | Owner | Rename a floor. |
| DELETE | `/floor-maps/:floorMapId/delete` | Owner | Delete a floor. |

## Room types and rooms

| | Endpoint | Who | What it does and why |
|---|---|---|---|
| POST | `/room-types` | Owner | Create a room type. |
| GET | `/room-types` | Any worker | Room types. |
| GET | `/room-types/:roomTypeId` | Any worker | One room type. |
| PATCH | `/room-types/:roomTypeId` | Owner | Edit a room type. |
| DELETE | `/room-types/:roomTypeId/delete` | Owner | Delete a room type. |
| POST | `/rooms` | Owner | Create a room on a floor. |
| GET | `/rooms` | Any worker | Rooms. |
| GET | `/rooms/:roomId` | Any worker | One room with seats. |
| PATCH | `/rooms/:roomId` | Owner | Edit a room. |
| PATCH | `/rooms/:roomId/status/:status` | tables.manage | Room available / occupied / cleaning ... |
| DELETE | `/rooms/:roomId/delete` | Owner | Delete a room. |

## Seats

| | Endpoint | Who | What it does and why |
|---|---|---|---|
| POST | `/seats` | Owner | Place a table / seat (x, y on the map). |
| GET | `/seats` | Any worker | Seats. |
| GET | `/seats/:seatId` | Any worker | One seat. |
| PATCH | `/seats/:seatId` | Owner | Move or rename a seat. |
| PATCH | `/seats/:seatId/status/:status` | tables.manage | Seat available / occupied / reserved ... |
| DELETE | `/seats/:seatId/delete` | Owner | Delete a seat. |

# 6. Realtime events

Connect with Socket.IO to `/realtime`: signed-in apps send `auth: { token }`, devices that are not signed in send `auth: { deviceId, deviceSecret }`. The server puts each socket in the right rooms, so it only receives what it may see.

| Event | From → to | Why |
|---|---|---|
| `order.created / order.updated` | Server → the order's worker, owners, orders.view_all, admins | Kitchen screens, tills and dashboards update without refreshing. |
| `device.requested / updated / removed` | Server → owners and devices.manage | Approve new devices the moment they ask. |
| `device.presence` | Server → owners and devices.manage | Online / offline dot per device. |
| `device.status` | Server → the device itself | Status, type, paired till and MoMo code; the app switches screen live. |
| `display.preview` | Till → its customer screen (relayed) | Instant feedback (~10 ms) when the cashier taps or scans, with the cashier name. |
| `display.cart / order / paid / cleared` | Server → paired customer screen | Saved cart, amount to pay, paid confirmation, clear. Only customer-safe fields. |
| `permissions.changed` | Server → that user | New permissions applied without signing in again. |
| `session.revoked` | Server → that user / device | Deactivated, signed in elsewhere, or device disabled: the app logs out. |

### Rules for app developers

- Treat events as "something changed": update the screen from the payload or refetch.
- On `session.revoked`, log out and show the reason.
- On `permissions.changed` (or a new `X-Permissions-Version` header), refetch `/auth/me` and redraw menus.
- Customer screens never sign in; they listen with the device secret and render `display.*` and `device.status`.

## 7. What already makes it strong

These are real selling points compared with simple POS apps. Mention them in demos and marketing.

**Live permissions**
Role or permission changes reach open apps instantly; deactivating a worker signs them out everywhere in the same second.

**Safe delegation**
Managers can give access onward, but never more than they have, never to themselves, never over someone stronger.

**Device-bound security**
Only approved devices work; each has a secret; workers are on one device at a time; disabling a device ends its sessions.

**Device roles**
Till, kitchen screen and customer screen each only do their job, enforced by the server, switchable live.

**Instant customer screen**
Taps show on the paired customer screen in ~10 ms, with the real cashier name and the device's MoMo code.

**Full history**
Every change records who, when and before / after. History survives deletes.

**Exact money**
Numeric columns and cent-level checks; split and credit payments with permissions.

**Multi-business**
Every query is scoped to the company; workers see only their own carts and orders unless allowed.

# 8. Good to have: speed and safety

Improvements to what exists. Most are small and make the biggest difference to how fast and reliable the product feels.

## Speed

| Improvement | Why | How | Impact | Effort |
|---|---|---|---|---|
| **Run the API next to the database** | Each database trip from Rwanda to the US database costs ~300 ms; adding an item needs several. | Host the API in the same region as the database (or move the database close to the API). Expected: add-item from ~3.5 s to well under 200 ms. | Very high | Low |
| **Pagination and date filters** | GET /orders and GET /carts return every row; they slow down as the business grows. (Audit logs are already paged.) | Add ?limit, ?cursor (createdAt,id) and ?from / ?to, like /audit-logs. Indexes on (company_id, createdAt) already exist. | High | Low |
| **Cache the catalog on the till** | Products rarely change but every till loads them all. | Keep a per-company catalog version; tills cache products and only refetch when the version changes (or ETag / 304). | High | Medium |
| **Gzip responses** | Order and product lists are large JSON. | Enable compression middleware (or at the proxy). | Medium | Low |
| **Redis** | Login lock-outs and socket rooms live in one process. | Redis for login throttling, the Socket.IO adapter (several API instances) and short caches. | Medium | Medium |
| **Reports from summaries** | Reports scan all orders. | Daily summary table (per company, product, employee, payment method) filled when orders complete; or a read replica. | Medium | Medium |

## Safety and reliability

| Improvement | Why | How | Impact | Effort |
|---|---|---|---|---|
| **Refresh tokens** | Only a 1-day access token today (refresh is commented out in the config). | Short access token (15 min) + rotating refresh token stored hashed; revoke on logout / deactivate. | High | Medium |
| **Idempotency on payments** | A retried "complete" or "checkout" on a bad network could double-submit. | Accept an Idempotency-Key header on checkout / complete and return the first result for repeats. | High | Low |
| **Rate limiting and headers** | Only login is rate-limited; the socket allows any origin. | @nestjs/throttler globally, helmet headers, restrict CORS to your app domains. | High | Low |
| **Health, logs, errors** | No health endpoint or error tracking. | GET /health (DB check) for the host, structured logs, Sentry; automatic daily database backups. | Medium | Low |
| **Tests in CI** | The end-to-end checks used during development live outside the repo. | Move them into test/ (jest-e2e) and run on every push with a test database. | Medium | Medium |

# 9. Future features

New capabilities that make the product more useful and harder to replace. "Unique" marks features most competitors in the region do not offer well. Several already have permission keys reserved, so the access model is ready for them.

| Feature | Why | What to build | Value |
|---|---|---|---|
| **Offline mode for tills** | Internet drops are common; a till that stops selling loses money. | Till keeps catalog and a queue of sales locally, syncs when back online (idempotency keys make replays safe). A strong reason to choose this POS. | Unique |
| **MoMo / Airtel Money payments** | The MoMo code is shown, but payment is confirmed by hand. | Integrate MTN MoMo (request-to-pay) and Airtel Money APIs: the customer gets a prompt on their phone, the order completes automatically. QR code on the customer screen. | Unique |
| **RRA EBM receipts** | VAT-registered businesses in Rwanda must issue EBM receipts. | Integrate with the RRA EBM system so every sale produces a valid tax receipt. Check current RRA requirements first. | Unique |
| **Inventory (reserved: inventory.*)** | Owners want to know stock and theft. | Stock per product, deduct on sale, adjustments with reasons, low-stock alerts, suppliers and purchases, profit = selling - buying price. | High |
| **Credit book (reserved: credits.view)** | Credit sales are stored but there is no list or repayment. | Customers who owe, how much, since when; record repayments; SMS reminders. | High |
| **Cash-up and shifts (reserved: finance.verify)** | No end-of-day check of money. | Open / close shift per till, expected vs counted cash and MoMo, differences flagged for the owner. | High |
| **Receipts** | Customers ask for proof of payment. | Thermal printer (ESC/POS) receipts and kitchen tickets; SMS / WhatsApp digital receipt. | High |
| **Reports dashboard** | Only staff performance today. | Sales by day / hour, top products, payment methods, profit, cancellations; export CSV / PDF; daily summary by SMS / email to the owner. | High |
| **Discounts, taxes, refunds** | Tax is always 0; no discounts or refunds. | Tax rates per product, discounts with a permission, refunds / returns linked to the original order. | Medium |
| **Restaurant extras** | Ingredients exist; no modifiers or table bills. | Modifiers ("no onions"), kitchen stations, split bill per seat, tips, table reservations. | Medium |
| **Hotel extras (reserved: services.manage)** | Rooms exist; no stays. | Reservations, check-in / check-out, charging room services to the stay. | Medium |
| **Customers and loyalty** | No customer records. | Customer by phone number, purchase history, points, promotions on the customer screen when idle. | Medium |
| **Multi-branch** | One company = one place today. | Branches under a company with shared catalog, per-branch stock and reports. | Medium |
| **Languages** | Staff and customers speak Kinyarwanda, English, French, Swahili. | Translatable messages and customer-screen language per device. | Unique |

## 10. Suggested roadmap

### Now (1–2 weeks)

- Host API and database in the same region
- Pagination + date filters
- Idempotency keys on checkout / complete
- Rate limiting, helmet, CORS allow-list
- Health endpoint, error tracking, backups
- Commit e2e tests and run them in CI

### Next (1–2 months)

- Refresh tokens
- Receipts (printer + SMS / WhatsApp)
- Reports dashboard + daily owner summary
- Credit book and repayments
- Shifts and cash-up
- Catalog caching on tills

### Later (differentiators)

- Offline mode for tills
- MoMo / Airtel Money payment APIs + QR
- RRA EBM integration
- Inventory and suppliers
- Restaurant / hotel extras, loyalty
- Multi-branch, languages